

YUEDMAI Smart Stretch Coach Technical Report

Generated for the `stretch_applab/` and `arduino/` workspace.

Date: 2026-05-28

Executive Summary

YUEDMAI is a local-first smart stretching kiosk prototype for Arduino UNO Q. The system combines a local FastAPI web app, USB or phone camera streaming, pose tracking, browser-based session UI, optional Arduino App Lab hardware controls, and separate Arduino firmware for a wearable Nano IMU node plus a UNO Q fusion hub.

The key design choice is that all normal runtime behavior is local. The UNO Q or local Python runtime hosts the web app, serves the kiosk UI, streams camera frames, runs pose tracking, accepts Nano IMU telemetry, and exposes state through local HTTP and WebSocket endpoints. The Arduino sketches use newline-delimited JSON, BLE characteristics, Serial, Arduino RouterBridge callbacks, and Modulino hardware to add physical input and feedback. [R01] [R02] [R03] [R15] [R16]

Scope And Source Basis

This report covers:

- `stretch_applab/`: App Lab packaging, Python FastAPI app, camera sources, pose tracking, browser UI, hardware bridge, Nano BLE subscriber, App Lab MCU sketch, models, and docs.
- `arduino/`: Arduino IDE firmware sketches, serial protocol documentation, Nano signal dashboard, and firmware worklog.
- Local source references are listed in the appendix. Most claims below are derived directly from code and docs in this workspace, with citations such as [R06].

System At A Glance

Area	What is used	Main purpose	Main source
App runtime	Python, FastAPI, Uvicorn	Local kiosk server and API	[R01], [R06]
Deployment target	Arduino UNO Q Linux / Arduino App Lab	Runs local app at 0.0.0.0:8000 by default	[R01], [R02]
Browser UI	HTML templates, CSS, vanilla JS	Landing, setup, session, phone sender, dashboard, summary pages	[R06], [R13]
USB camera	OpenCV cv2.VideoCapture	Preferred camera source in auto mode	[R08]
Phone camera	Browser getUserMedia, canvas JPEG, WebSocket	Camera fallback through QR pairing	[R06], [R13]
Video stream	MJPEG over HTTP /video_feed	Kiosk live camera feed	[R06]
Pose model	MediaPipe Pose Landmarker .task	Default body landmark extraction	[R04], [R09]
Pose fallback	MoveNet TFLite / LiteRT	Lightweight fallback if MediaPipe unavailable	[R04], [R09]
Experimental pose backend	NCNN YOLO pose, Vulkan optional	GPU-oriented pose backend for UNO Q experiments	[R04], [R09]
Wearable	Arduino Nano 33 BLE Sense Lite	Forearm IMU and optional onboard sensor node	[R15], [R17]
Fusion hub	UNO Q Arduino sketch	Fuses camera pose, Nano IMU, distance, buttons	[R16], [R18]

Area	What is used	Main purpose	Main source
App Lab MCU controls	Arduino RouterBridge, Modulino Knob/Buttons/Pixels	Physical navigation and LED feedback for kiosk pages	[R14]
Protocols	HTTP, WebSocket, MJPEG, BLE, Serial, Web Serial, RouterBridge	Local data transport between browser, Python, and Arduino	[R06], [R13], [R14], [R16]

Main Runtime Architecture

The app starts from `stretch_applab/python/main.py`, which sets pose defaults and launches Uvicorn against `app.main:app`. The FastAPI app is in `stretch_applab/python/app/main.py`. On startup it:

- Logs runtime settings.
- Starts the hardware bridge.
- Starts the Python Nano BLE manager.
- Starts the camera source manager. [R06] [R12]

The FastAPI app owns:

- Page routes: `/`, `/setup`, `/session`, `/dashboard`, `/phone`, `/summary/{session_id}`.
- Stream routes: `/video_feed`, `/ws/phone-frame`, `/ws/hardware`.
- Status and health routes: `/api/status`, `/api/health`, `/api/hardware`.
- Session controls: `/api/session/start`, `/api/session/pause`, `/api/session/next`, `/api/session/reset`, `/api/session/config`.
- Nano and hardware bridge APIs: `/api/nano_imu`, `/api/hardware/feedback`.
- Capture/sharing endpoints: `/api/session/captures`, `/summary/{session_id}/image/{index}.{ext}`, `/summary/{session_id}/download.zip`, QR PNG endpoints. [R06]

High-Level Data Flow

1. The user opens the kiosk UI on the UNO Q local URL, usually `http://<UNO_Q_IP>:8000/`.
2. The app selects a camera source. auto mode prefers USB and falls back to phone QR. usb forces USB. phone forces the phone sender flow. [R03] [R07]
3. Camera frames enter the Python app either from OpenCV USB capture or from phone JPEG frames sent over WebSocket.
4. `/video_feed` pulls the latest frame, runs `inference.process_frame()`, optionally draws pose overlays, encodes JPEG, and serves MJPEG frames. [R06] [R09]
5. `/api/status` aggregates camera status, session status, pose status, hardware bridge status, Nano BLE status, capture status, and setup boundary status. The session page polls this every 500 ms. [R06] [R13]
6. The browser session page updates the HUD, boundary check, audio cues, progress, score display, capture summary, and hardware feedback.
7. Optional Nano IMU telemetry can reach Python over BLE directly, through the UNO Q App Lab RouterBridge, through HTTP `/api/nano_imu`, or into the standalone UNO Q fusion sketch over Serial/BLE. [R11] [R12] [R14] [R16]
8. Optional Arduino firmware fuses camera pose JSON, Nano IMU JSON, distance readings, and button input into `stretch_state` JSON at 10 Hz. [R16] [R18]

Deployment And Runtime Configuration

The packaged app is meant to be copied into an Arduino App Lab app folder with `python/`, `sketch/`, `docs/`, and `README.md`. The app launcher defaults to a local Uvicorn server. [R01]

Important environment variables:

- `APP_HOST`: default `0.0.0.0`.
- `APP_PORT`: default `8000`.
- `PUBLIC_BASE_URL`: optional override for QR URLs and share URLs.
- `FORCE_CAMERA_MODE`: `auto`, `usb`, or `phone`.
- `POSE_TRACKING_ENABLED`: enables/disables pose tracking.
- `POSE_BACKEND`: `mediapipe`, `movenet`, or `ncnn_pose`.
- `POSE_MODEL_PATH`: default `models/pose_landmarker.task`.
- `POSE_DELEGATE`: `cpu` or `gpu` for MediaPipe.
- `POSE_INFERENCE_WIDTH`: default configured by code/docs, used to downsize pose inference.
- `POSE_FRAME_STRIDE`: reuse pose results every N frames.
- `POSE_ASYNC_ENABLED`: default `true`.
- `POSE_MAX_ASYNC_FPS`: caps async pose worker submission.
- `POSE_FALLBACK_BACKEND`: default `movenet`.
- `MOVENET_MODEL_PATH`, `MOVENET_INPUT_SIZE`, `MOVENET_NUM_THREADS`.
- `NCNN_MODEL_DIR`, `NCNN_INPUT_SIZE`, `NCNN_USE_VULKAN`, `NCNN_GPU_INDEX`, `NCNN_CONFIDENCE`, `NCNN_IOU`.
- `NANO_BLE_ENABLED`, `NANO_BLE_NAME`, `NANO_BLE_SERVICE_UUID`, `NANO_BLE_IMU_CHAR_UUID`, `NANO_BLE_SCAN_TIMEOUT_SEC`, `NANO_BLE_RETRY_SEC`. [R01] [R03] [R09] [R12]

Python Backend Stack

Python dependencies in `requirements.txt`:

- `fastapi`: web app and API framework.
- `uvicorn[standard]`: ASGI server.
- `opencv-python-headless`: camera capture, JPEG decode/encode, overlays.
- `numpy`: image arrays and geometry math.
- `qrcode`: phone pairing and summary QR image generation.
- `pillow`: image support for QR/image handling.
- `jinja2`: HTML template rendering.
- `python-multipart`: form/file related FastAPI support.
- `ai-edge-litert`: TFLite/LiteRT runtime for MoveNet.
- `bleak`: Python BLE scanner/client for Nano IMU notifications. [R05]

Extra MediaPipe environment dependencies in `requirements-mediapipe.txt`:

- `numpy<2`.
- `opencv-contrib-python==4.11.0.86`.
- `mediapipe==0.10.15`. [R05]

Camera Inputs

USB Camera

The USB source scans indexes 0 through 5, opens each candidate with OpenCV, requests 640x480, requests 15 FPS, sets buffer size to 1, validates by reading up to five frames, then stores the latest successful frame. On Windows it tries `cv2.CAP_DSHOW` first. Repeated read failures mark the camera disconnected. [R08]

USB is preferred when `FORCE_CAMERA_MODE=auto`. If USB disappears in auto mode, the source manager falls back to phone QR mode. [R07]

Phone Camera

The phone page is served at `/phone`. It:

- Requests browser camera access with `navigator.mediaDevices.getUserMedia`.
- Uses environment-facing camera preference where available.
- Draws the video to a hidden canvas at 640x480.
- Converts each frame to JPEG with quality 0.65.
- Sends binary JPEG data over WebSocket to `/ws/phone-frame`.
- Targets 10 FPS.
- Reconnects the WebSocket if it closes. [R06] [R13]

The Python side accepts the WebSocket, tracks phone clients, decodes JPEG bytes with OpenCV `cv2.imdecode`, updates the latest frame store, and records frame/decode counters. [R06] [R08]

Video Output

The kiosk browser displays `/video_feed` as an MJPEG stream. The generator:

- Calls `full_status()`.
- Gets the latest frame from the selected camera source.
- Builds a placeholder frame if no camera frame is available.
- Runs `inference.process_frame()` with source, FPS, session state, score, and overlay flags.
- Encodes the processed frame as JPEG quality 75.
- Yields multipart `image/jpeg` frames.
- Sleeps about 0.08 seconds between loop iterations. [R06]

The live frame output is intentionally separate from `/api/status` polling. The image stream carries pixels; the status endpoint carries structured state. [R02] [R06]

Pose Tracking Pipeline

The pose pipeline is local and frame-based. The default backend is MediaPipe Pose Landmarker using `models/pose_landmarker.task`; the app disables segmentation masks and tracks one pose. [R04] [R09]

Main steps:

1. The selected camera frame reaches `/video_feed`.
2. `inference.process_frame()` copies the frame.
3. `PoseTracker.process()` either submits the frame to an async worker or runs detection inline.
4. The active backend produces landmarks.
5. Landmarks are mapped to the YUEDMAI names: shoulders, elbows, wrists, hips, knees, ankles.
6. `compute_pose_flags()` derives simple booleans and confidence.

7. If enabled, the overlay draws limb lines, landmark dots, and small status labels.
8. `get_pose_status()` exposes the latest pose status to `/api/status`. [R09]

Pose Backends

MediaPipe backend:

- Default model path: `models/pose_landmarker.task`.
- Default delegate: CPU, with optional GPU request and CPU fallback if GPU fails.
- Running mode: video.
- `num_poses=1`.
- Minimum detection, presence, and tracking confidence: `0.5`.
- Segmentation masks disabled. [R04] [R09]

MoveNet fallback:

- Default model path: `models/movenet_lightning.tflite`.
- Default input size: 192.
- Default threads: 2.
- Runtime search order: `ai_edge_litert`, `tflite_runtime`, then TensorFlow Lite through TensorFlow.
- Maps 17 COCO keypoints into the same YUEDMAI landmark names. [R04] [R09]

NCNN YOLO pose backend:

- Default model dir: `models/yolov8n-pose_ncnn_model`.
- Default input size: 320.
- Confidence threshold: `0.25`.
- IoU threshold: `0.45`.
- GPU index: 0.
- Optional Vulkan compute.
- Expects YOLO pose NCNN output rows shaped like box confidence plus 17 keypoints. [R04] [R09]

Pose Flags

The current MVP flags are:

- `pose_ok`: landmarks exist and confidence is above zero.
- `user_visible`: both shoulders and both hips visible.
- `upper_body_visible`: both shoulders plus at least one elbow and one wrist visible.
- `full_body_visible`: shoulders, hips, knees, and ankles visible.
- `arm_raised`: either wrist is above the same-side shoulder by `0.08` normalized y units.
- `torso_centered`: shoulder center x and hip center x differ by less than `0.12`.
- `confidence`: average landmark confidence for upper-body landmarks.

The code uses `MIN_LANDMARK_CONFIDENCE=0.5`. [R04] [R09]

Setup Boundary Check

The session page starts with a boundary precheck when coming from setup. It waits for camera and pose status to become ready, then requires the ready condition to hold for 450 ms before automatically starting the session. [R06]

[R13]

Boundary status depends on:

- Camera state: active, waiting for phone, or no camera.
- Pose tracker availability and model load state.
- Pose freshness of two seconds or less.
- `user_visible`.
- `upper_body_visible` or `full_body_visible`.
- Confidence threshold of about 0.28 for the setup pass.

Possible setup states include `NO_CAMERA`, `WAITING_FOR_CAMERA`, `POSE_UNAVAILABLE`, `CHECKING_BOUNDARY`, `STEP_INTO_FRAME`, `SHOW_UPPER_BODY`, `LOW_CONFIDENCE`, and `READY_TO_START`. [R06]

Session Flow

The FastAPI session manager is a small state machine for the hackathon UI. Timing constants:

- Ready countdown: 5 seconds.
- Stretch segment: 30 seconds.
- Rest segment: 10 seconds. [R10]

Configuration:

- Mode: before or after.
- Body focus: upper, lower, or full.
- Duration: 3, 5, or 8 minutes.

Routine selection changes based on mode and body focus. Examples include arm circles, hip opener, hamstring sweep, wall slides, overhead reach hold, standing side bend stretch, doorway chest opener, shoulder external rotation, quad stretch, hamstring stretch, calf stretch, and shoulder stretch. [R10]

Session states surfaced to the browser:

- `IDLE`
- `READY`
- `REST`
- `HOLD`
- `GOOD`
- `DONE`
- `NO_CAMERA`
- `WAITING_FOR_PHONE` [R10]

Browser behavior:

- Polls `/api/status` every 500 ms.
- Posts session actions to `/api/session/{action}`.
- Updates HUD text, timer, progress bar, score, audio, and overlays.
- Captures still photos from the MJPEG image element when the state becomes `GOOD` or `DONE`.
- Uploads captures to `/api/session/captures`.
- Shows final summary QR/download/share info.

- Auto-advances to the next stretch after intermediate DONE states. [R06] [R13]

Scoring And Stretch Models

There are three scoring layers in the workspace.

Browser/Python Session Score

The main app score starts from session phase timing and then adjusts based on pose and Nano metrics:

- HOLD: base score rises from about 64.
- GOOD: base score rises from about 84.
- DONE: base score 100.
- Pose penalties apply for low confidence, missing user, torso not centered, missing full body when required, or missing arm raise when required.
- Nano penalties/bonuses apply when fresh Nano metrics exist, including arm raised, stable, `stability_score`, and high `gyro_mag`. [R10]

Stretch-Specific Models

`stretch_models.py` implements:

- `hamstring_reach`: uses knee angle, hip fold angle, reach distance to front ankle, and optional Nano stability.
- `side_bend`: uses overhead reach, lateral tilt, elbow extension, side reach, and optional Nano arm/stability scores.

Both models return availability, model name, score, success flag, feedback text, and metrics. [R10]

UNO Q Firmware Score

The standalone UNO Q fusion sketch emits a prototype score only during a started and unpaused session:

- 40 points if user is visible and distance is OK.
- 30 points if Nano says arm is raised.
- 20 points if camera says arm is raised.
- 10 points if Nano says stable.
- Clamped to 0..100; null during setup. [R16] [R18]

Browser UI And Hardware Events

The browser app uses `shared hardware.js` for hardware input:

- Connects to `/ws/hardware`.
- Receives JSON events of type `hardware_event`.
- Emits DOM event `YUEDMAI:hardware`.
- Receives Nano IMU events and emits DOM event `YUEDMAI:nano-imu`.
- Falls back to keyboard controls: arrows for prev/next, Enter/Space for confirm, Escape/Backspace for back, n for alternate action.
- Sends page feedback through `/api/hardware/feedback`. [R11] [R13]

Action normalization maps:

- Knob left/right to PREV/NEXT.
- Knob press and button A to CONFIRM.

- Button B to BACK.
- Button C to ALT.
- Long versions to long actions. [R11] [R13] [R14]

Capture And Sharing Output

The capture system is in memory:

- The app creates a 12-character session id.
- Browser uploads base64 data URLs for JPEG/PNG/WebP captures.
- Each image is capped at 6 MB.
- Captures store index, name, score, extension, MIME type, bytes, and timestamp.
- Summary page serves images and ZIP download.
- QR endpoints generate phone camera and summary QR PNGs.
- Restarting the Python process clears in-memory captures. [R02] [R06]

App Lab MCU Sketch In `stretch_applab/sketch`

The App Lab sketch is the MCU-side companion for the packaged app. It uses:

- `Arduino_RouterBridge` when available.
- `Arduino_Modulino` or legacy `Modulino.h` when available.
- Optional `ArduinoBLE`, but `USE_NANO_BLE_IMU` is currently 0 by default because BLE central scanning can stall `RouterBridge/button` handling. [R14]

Inputs:

- Modulino Knob rotation.
- Modulino Knob press and long press.
- Modulino Buttons A/B/C with short and long press.
- Optional Nano BLE packets if enabled. [R14]

Outputs:

- `RouterBridge hardware_event` notifications to Python.
- `RouterBridge nano_imu` notifications if Nano BLE forwarding is enabled.
- Modulino Pixels and button LEDs based on browser feedback.
- Serial debug logs at 115200 baud. [R14]

Timing/constants:

- Input poll: 25 ms.
- Long press: 850 ms.
- Feedback refresh: 80 ms.
- BLE poll interval: 250 ms.
- BLE scan retry: 10000 ms.
- Pixel count: 8. [R14]

Arduino Folder: Hardware Roles

The standalone `arduino/` folder has two sketches:

- `NanoStretchNode`: wearable forearm IMU node on Arduino Nano 33 BLE Sense Lite.
- `UnoQStretchHub`: UNO Q hardware hub for distance, buttons, buzzer, pixels, optional LCD, serial input, BLE Nano input, and fusion state.

The Arduino docs explicitly state that camera inference does not run in `.ino` files. Camera pose comes from the Linux/Python app as compact JSON. [R15] [R16]

NanoStretchNode Firmware

The Nano is worn on the forearm, with consistent board orientation between calibration and use. It estimates forearm angle and stability locally, then publishes compact or rich telemetry. [R15] [R17]

Nano Libraries

Core:

- `Arduino.h`
- `math.h`
- `stdlib.h`
- `string.h`
- One IMU backend: `Arduino_LSM9DS1` or `Arduino_BMI270_BMM150`

Optional/onboard:

- `Arduino_APDS9960` for proximity/light/color/gesture.
- `Arduino_LPS22HB` for barometric pressure.
- `Arduino_HTS221` or `Arduino_HS300x` for non-Lite temperature/humidity variants.
- PDM for microphone level.
- `ArduinoBLE` for BLE telemetry and commands. [R15] [R17]

Temperature and humidity are disabled by default because Nano 33 BLE Sense Lite does not include that sensor. [R15] [R17]

Nano Sensors And Signals

Primary IMU signals:

- Accelerometer: `ax`, `ay`, `az`.
- Gyroscope: `gx`, `gy`, `gz`.
- Smoothed pitch and roll.
- `relative_pitch`: pitch minus calibration baseline.
- `gyro_mag`: instantaneous gyroscope magnitude.
- `gyro_avg`: smoothed gyroscope magnitude.
- `stability_score`: prototype 0-100 stillness score.

Optional signals:

- Magnetometer: `mx`, `my`, `mz`, `mag_mag`, `heading_deg`, `mag_ok`.
- APDS9960: `proximity`, `red`, `green`, `blue`, `ambient`, `gesture_code`, `gesture`, `apds_ok`.
- Barometer: `pressure_kpa`, `pressure_hpa`, `baro_ok`.

- Microphone: mic_rms, mic_peak, mic_avg_abs, mic_dbfs, mic_level, mic_samples, mic_ok. [R16] [R17] [R19]

Nano Classification

Constants:

- Serial baud: 115200.
- Compact JSON interval: 50 ms or 20 Hz.
- Full JSON interval: 100 ms or 10 Hz.
- BLE compact interval: 100 ms or 10 Hz.
- Calibration duration: 2000 ms.
- IMU stale timeout: 1000 ms.
- Stable dwell: 300 ms.
- Default arm threshold: 55 deg.
- Default stability threshold: 20 deg/s.
- EMA alpha: 0.18. [R17]

States:

- NANO_ARM_LOW
- NANO_ARM_RAISED
- NANO_HOLD_STABLE
- NANO_UNSTABLE
- NANO_CALIBRATING
- NANO_ERROR [R16] [R17]

Logic:

- Calibration averages smoothed pitch during the first two seconds.
- $\text{relative_pitch} = \text{smoothedPitchDeg} - \text{baselinePitchDeg}$.
- $\text{arm_raised} = \text{abs}(\text{relative_pitch}) \geq \text{armRaisedThresholdDeg}$.
- $\text{stableHold} = \text{smoothedGyroMagDps} \leq \text{stabilityThresholdDps}$.
- If arm is raised and stable for 300 ms, state becomes NANO_HOLD_STABLE. [R17]

Nano Commands

Plain newline-delimited commands:

```
CALIBRATE
STATUS
SET_ARM_THRESHOLD 55
SET_STABILITY_THRESHOLD 20
PLOTTER_ON
PLOTTER_OFF
OUTPUT_JSON
OUTPUT_FULL_JSON
OUTPUT_PLOTTER
```

OUTPUT_JSON is compact UNO-compatible telemetry. OUTPUT_FULL_JSON adds optional sensor fields. OUTPUT_PLOTTER produces labelled numeric lines for Arduino Serial Plotter. [R16] [R17]

Nano BLE Protocol

Device name:

YUEDMAI-NanoIMU

UUIDs:

Service: 19b10000-e8f2-537e-4f6c-d104768a1214
 IMU notify: 19b10001-e8f2-537e-4f6c-d104768a1214
 Command write: 19b10002-e8f2-537e-4f6c-d104768a1214

The Nano advertises the service, notifies compact JSON on the IMU characteristic, and accepts commands such as CALIBRATE on the command characteristic. [R16] [R17]

Compact BLE payload fields include:

- type: "nano_imu"
- t
- relative_pitch
- gyro_mag
- stability_score
- arm_raised
- stable
- state
- heading_deg
- mag_mag
- mag_ok [R16]

Python can subscribe directly through bleak using NanoBLEManager, which scans by service and then by device name, subscribes to the IMU characteristic, decodes UTF-8 JSON, and publishes it into the hardware bridge. [R12]

There is also a helper script, nano_ble_forwarder.py, that subscribes to BLE notifications and posts parsed JSON to http://127.0.0.1:8000/api/nano_imu. [R12]

UnoQStretchHub Firmware

The standalone UNO Q hub sketch receives:

- Nano wearable IMU JSON over BLE or Serial1.
- Forwarded Nano JSON over USB Serial from the Linux/Python app.
- Camera pose JSON from the Linux/Python app.
- Distance from Modulino Distance or mock distance.
- Buttons from Modulino Buttons.

It emits:

- stretch_state JSON over USB Serial at 10 Hz.
- Pixel colors through Modulino Pixels.
- Tones through Modulino Buzzer.
- Optional LCD output if a library is added later.
- Debug and acknowledgement lines starting with #. [R16] [R18]

UNO Q Libraries And Feature Flags

Feature flags:

- `USE_NANO_ON_SERIAL1` 1
- `USE_NANO_FORWARD_FROM_USB_SERIAL` 1
- `USE_NANO_ON_BLE` 1
- `USE_MODULINO_DISTANCE` 1
- `USE_MODULINO_PIXELS` 1
- `USE_MODULINO_BUZZER` 1
- `USE_MODULINO_BUTTONS` 1
- `USE_LCD` 0
- `USE MOCK_DISTANCE` 0
- `AUTO MOCK_DISTANCE_IF_LIBRARY_MISSING` 1 [R18]

Libraries:

- `Arduino.h`
- `math.h`
- `stdlib.h`
- `string.h`
- Optional `Arduino_Modulino.h` or legacy `Modulino.h`
- Optional `ArduinoJson`
- Optional `ArduinoBLE` [R15] [R18]

The fallback JSON parser supports the simple flat messages documented for this prototype if `ArduinoJson` is missing. [R15] [R18]

UNO Q Fusion Constants

- USB Serial baud: 115200.
- Nano Serial baud: 115200.
- Output interval: 100 ms or 10 Hz.
- BLE scan retry: 2500 ms.
- Distance interval: 100 ms.
- Nano stale timeout: 1000 ms.
- Camera stale timeout: 1000 ms.
- Distance stale timeout: 1000 ms.
- State debounce: 300 ms.
- Bad-form reset: 1200 ms.
- Good-to-done transition: 1500 ms.
- Default distance range: 80 . . 220 cm.
- No-user distance max: 300 cm.
- Camera confidence minimum: 0.50.
- Stability threshold: 20 deg/s.
- Target hold duration: 8 sec. [R18]

UNO Q Fusion State Machine

Final states:

- NO_USER
- STEP_BACK
- STEP_CLOSER
- READY
- RAISE_ARM
- HOLD_STEADY
- UNSTABLE
- GOOD
- DONE
- SENSOR_ERROR [R16] [R18]

Core hold condition requires:

- Session started and not paused.
- Nano data fresh and valid.
- Camera frame good.
- Distance within range.
- Nano stable.
- Nano arm raised.
- Camera arm raised.
- Torso centered.
- Nano gyroscope magnitude below threshold. [R18]

The state machine checks stale sources, distance guidance, camera visibility/confidence, session status, Nano health, routine done flags, arm raise status, shakiness, and hold success. [R18]

UNO Q Serial Commands

Plain commands:

```
START
PAUSE
NEXT
RESET
CALIBRATE_NANO
SET_MODE before
SET_MODE after
SET_BODY_FOCUS upper
SET_BODY_FOCUS lower
SET_BODY_FOCUS full
SET_TARGET_HOLD 8
SET_DISTANCE_MIN 80
SET_DISTANCE_MAX 220
STATUS
SET MOCK_DISTANCE 140
```

JSON commands:

```
{"type": "session_command", "command": "START"}
{"type": "config", "mode": "after", "body_focus": "full", "duration_min": 5}
```

CALIBRATE_NANO forwards CALIBRATE to Serial1 and to the BLE command characteristic when available. [R16]
[R18]

Serial JSON Data Schemas

All firmware messages are newline-delimited. Serial baud is 115200. Lines starting with # are human-readable logs/acknowledgements and should be ignored by machine receivers. [R16]

Camera Pose Input To UNO Q

Direction: Python camera app -> UNO Q firmware.

```
{
"type": "camera_pose",
"t": 12345,
"user_visible": true,
"full_body_visible": true,
"arm_raised": true,
"torso_centered": true,
"confidence": 0.82
}
```

The Arduino sketch does not run pose estimation. It trusts this as external input. [R15] [R16]

Nano IMU Input

Compact form:

```
{
"type": "nano_imu",
"relative_pitch": 62.0,
"gyro_mag": 7.0,
"stable": true,
"arm_raised": true,
"state": "NANO_HOLD_STABLE"
}
```

Full dashboard form adds raw IMU, magnetometer, APDS9960, barometer, microphone, thresholds, booleans, and state fields. [R16] [R17]

UNO Q Output

Direction: UNO Q firmware -> Python/dashboard/debug tools.

Rate: 10 Hz.

```
{
"type": "stretch_state",
"t": 12345,
"state": "HOLD_STEADY",
"instruction": "Hold the stretch",
"score": 72,
"distance_cm": 132.0,
"nano_angle": 61.2,
"gyro_mag": 8.1,
"camera_arm_raised": true,
"nano_arm_raised": true,
"hold_sec": 4.3,
"source_ok": true,
"nano_ok": true,
"camera_ok": true,
"distance_ok": true,
"session_started": true
}
```

The dashboard should use source_ok, nano_ok, camera_ok, and distance_ok as debug indicators. [R16]

Web Protocols And APIs

HTTP Pages

- GET /
- GET /setup
- GET /session

- GET /dashboard
- GET /phone
- GET /summary/{session_id} [R06]

HTTP APIs

- GET /api/status
- GET /api/health
- GET /api/hardware
- POST /api/nano_imu
- POST /api/hardware/feedback
- POST /api/rescan_usb
- POST /api/session/start
- POST /api/session/pause
- POST /api/session/next
- POST /api/session/reset
- POST /api/session/config
- POST /api/session/captures [R06]

Streams And Assets

- GET /video_feed: multipart MJPEG stream.
- GET /qr.png: phone pairing QR PNG.
- GET /summary_qr/{session_id}.png: summary QR PNG.
- GET /summary/{session_id}/image/{index}.{ext}: capture image.
- GET /summary/{session_id}/download.zip: ZIP of captures.
- WS /ws/phone-frame: binary JPEG frames from phone browser.
- WS /ws/hardware: JSON hardware and Nano events to browser. [R06]

Browser Local APIs

- navigator.mediaDevices.getUserMedia for phone camera.
- HTMLCanvasElement.toBlob for JPEG frame generation.
- WebSocket for phone frame upload and hardware event subscription.
- navigator.serial for Nano signal dashboard Web Serial connection.
- DOM custom events YUEDMAI:hardware and YUEDMAI:nano-imu. [R13] [R19]

Nano Signal Dashboard

arduino/tools/nano_signal_dashboard.html is a browser-based test dashboard for the Nano.

It uses:

- Web Serial navigator.serial.requestPort().
- Baud selection, typically 115200.
- Automatic OUTPUT_FULL_JSON command after connection.

- JSON parsing of nano_imu lines.
- Live charts/toggles for IMU, magnetometer, APDS9960, barometer, microphone, thresholds, booleans, and state.
- CSV export of sampled signals.
- Buttons for OUTPUT_FULL_JSON, OUTPUT_JSON, and CALIBRATE. [R19]

Outputs And Feedback Channels

Visual/browser outputs:

- Live camera stream with optional pose overlay.
- Setup and session UI pages.
- Status badges and debug panels.
- Boundary precheck guide.
- Rest/ready pose guideline media lookup under /static/PoseGuideline/.
- Summary capture slideshow, QR, share URL, ZIP download.
- Dashboard metrics and live Nano/session integration. [R06] [R13]

Physical outputs:

- Modulino Pixels for page/session feedback.
- Modulino button LEDs.
- Modulino Buzzer tones for hub state transitions.
- Optional LCD stubs for future state/instruction display. [R14] [R18]

Machine outputs:

- /api/status JSON.
- /api/health JSON.
- Hardware WebSocket JSON events.
- Nano IMU JSON events.
- UNO Q stretch_state JSON.
- CSV export from the Nano signal dashboard. [R06] [R11] [R16] [R19]

Error Handling And Resilience

Camera:

- USB reads are validated and repeated failures release the capture.
- Auto mode falls back to phone QR when USB is unavailable.
- Phone source tracks connected clients and decode errors.
- Placeholder frames keep /video_feed alive when no camera frame exists. [R06] [R07] [R08]

Pose:

- Missing MediaPipe model disables pose while preserving raw stream.
- MediaPipe GPU delegate failures fall back to CPU.
- MediaPipe can fall back to MoveNet if configured and available.
- Async pose processing keeps video responsive by drawing the last completed skeleton.

- Status includes model load state, backend, delegate, FPS, dropped/submitted/completed frames, and last error/warning. [R04] [R09]

Firmware:

- Malformed JSON is ignored.
- Unknown JSON fields are ignored.
- Unknown commands emit # WARN unknown command.
- Nano, camera, and distance data become stale after 1000 ms.
- Fallback flat JSON parsing exists if ArduinoJson is not installed.
- Mock distance can keep serial-only tests useful. [R16] [R18]

Privacy And Local-First Behavior

The docs describe the prototype as local-first: no external hosting, database, authentication, cloud service, or public web server is required for normal operation. Camera frames are processed locally and streamed locally. Captures are held in process memory, not a database. This lowers latency and avoids sending camera frames to a cloud service during the prototype flow. [R02] [R03]

Known Limitations

- This is a wellness prototype, not a medical system.
- The camera pose model is 2D and cannot fully detect depth, torso rotation, occlusion, or medical-quality form.
- The Nano pitch estimate is simple and accelerometer-based.
- Score is prototype wellness feedback, not medical assessment.
- Captures are in memory and disappear when the app restarts.
- Modulino behavior needs verification against the exact UNO Q and library version used.
- LCD support is stubbed until a specific display/library is selected.
- Camera pose flags are external inputs for Arduino sketches; no MediaPipe/OpenCV/ML inference runs in .ino files. [R02] [R04] [R15] [R16]

Practical Test Flow

App Test

1. Install Python requirements.
2. Start the app with Uvicorn or python/main.py.
3. Open `http://<UNO_Q_IP>:8000/`.
4. Test USB mode with `FORCE_CAMERA_MODE=usb`.
5. Test phone mode with `FORCE_CAMERA_MODE=phone`, open /setup, scan QR, and keep the phone sender page open.
6. Use `/session?debug=1` for camera/source/session debug fields.
7. Check `/api/status` and `/api/health`. [R01] [R03] [R06]

Nano Test

1. Upload `arduino/NanoStretchNode/NanoStretchNode.ino`.
2. Select the correct IMU backend.

3. Open Serial Monitor at 115200.
4. Keep the forearm still during the two-second calibration.
5. Verify nano_imu JSON.
6. Tune SET_ARM_THRESHOLD and SET_STABILITY_THRESHOLD.
7. Use the Nano signal dashboard for rich signal exploration. [R15] [R16] [R17] [R19]

UNO Q Hub Test

1. Upload arduino/UnoQStretchHub/UnoQStretchHub.ino.
2. Confirm Modulino and mock flags.
3. Open Serial Monitor at 115200.
4. Send START.
5. Send a camera_pose JSON line.
6. Send a nano_imu JSON line.
7. Expect READY, HOLD_STEADY, GOOD, and DONE sequence when distance and hold conditions are satisfied. [R15] [R16] [R18]

Reference Appendix

Ref	Source	Evidence used
R01	stretch_applab/README.md	App Lab package layout, launcher, run commands, environment variables
R02	stretch_applab/docs/SYSTEM_ARCHITECTURE.md	Main runtime components, local-first behavior, API surface, browser flows
R03	stretch_applab/docs/STRETCH_STREAM_README.md	Local kiosk behavior, camera modes, QR phone fallback, troubleshooting
R04	stretch_applab/python/docs/POSE_TRACKING.md	Pose backends, flags, tuning, limitations
R05	stretch_applab/python/requirements.txt; stretch_applab/python/requirements-mediapipe.txt	Python dependency inventory
R06	stretch_applab/python/app/main.py:43; :128-390; :393-543	Capture store, startup/shutdown, routes, status aggregation, boundary status, MJPEG
R07	stretch_applab/python/app/source_manager.py:13-158	Camera force modes, source selection, USB/phone status
R08	stretch_applab/python/app/camera_sources.py:15-171	USB camera constants, OpenCV capture, phone JPEG decode
R09	stretch_applab/python/app/inference.py; stretch_applab/python/app/pose_tracker.py; movenet_pose.py; ncnn_pose.py	Pose processing, flags, async worker, MediaPipe, MoveNet, NCNN
R10	stretch_applab/python/app/session_manager.py; stretch_applab/python/app/stretch_models.py	Routines, session timing, scoring, hamstring and side-bend models
R11	stretch_applab/python/app/hardware_bridge.py:20-298	Hardware event aliases, WebSocket fanout, Nano IMU cleaning, feedback bridge
R12	stretch_applab/python/app/nano_ble.py:19-159; stretch_applab/python/tools/nano_ble_forwarder.py	Python BLE scan/subscribe and HTTP forwarding

Ref	Source	Evidence used
R13	stretch_applab/python/app/templates/phone.html; stretch_applab/python/app/static/hardware.js; setup.js; session.js; dashboard.js	Browser camera sender, hardware DOM events, polling, captures, dashboard
R14	stretch_applab/sketch/sketch.ino:18-576	App Lab MCU controls, RouterBridge, Modulino inputs/outputs, optional Nano BLE
R15	arduino/README_ARDUINO.md	Hardware roles, wiring options, upload steps, libraries, integration notes
R16	arduino/SERIAL_PROTOCOL.md	Serial/BLE schemas, commands, output rates, states, errors, score
R17	arduino/NanoStretchNode/NanoStretchNode.ino:19-969	Nano firmware feature flags, libraries, sensors, calibration, output modes
R18	arduino/UnoQStretchHub/UnoQStretchHub.ino:20-1145	UNO Q hub flags, constants, fusion state machine, Serial/BLE/distance/buttons
R19	arduino/NANO_SIGNAL_DASHBOARD.md; arduino/tools/nano_signal_dashboard.html:688-795	Web Serial dashboard, full JSON command, CSV export